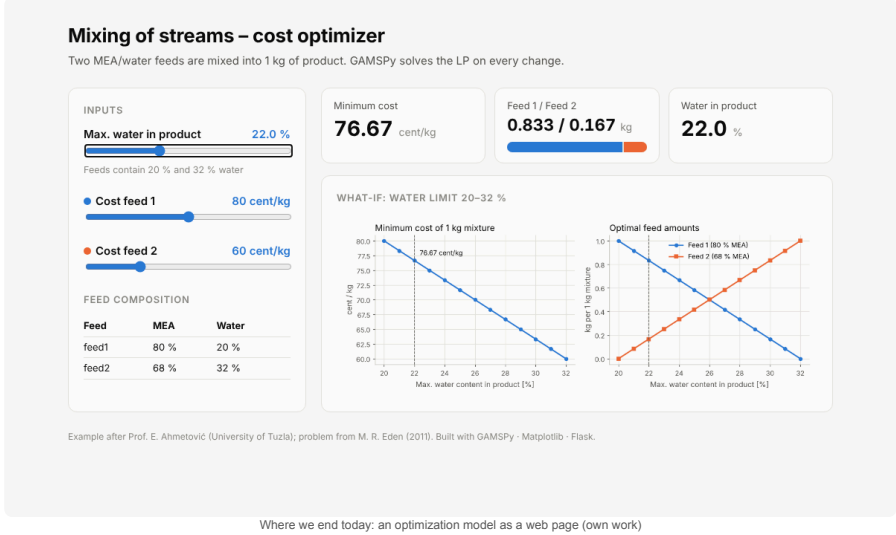
From GAMS to GAMSPy

Optimization modelling in Python – from model to interactive web page

Ing. Martin Necpal, PhD.

GUEST LECTURE
FACULTY OF MATERIALS SCIENCE AND TECHNOLOGY IN TRNAVA · STU BRATISLAVA

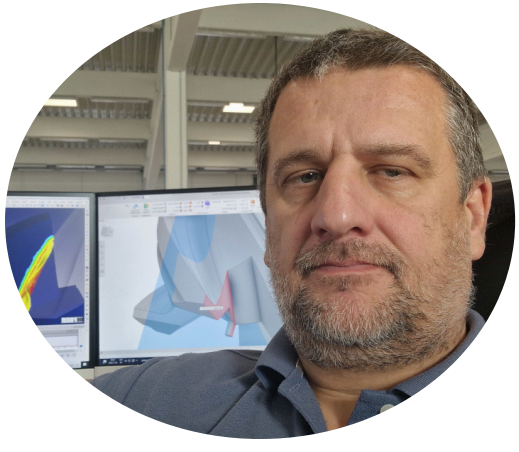


Where we end today: an optimization model as a web page (own work)

How to control the presentation (bar at the bottom)

- ← → or the ‹ › buttons – previous / next slide
- Dots – jump directly to a chosen slide
- ▶ or space – play / pause the spoken commentary
- **Auto** on – moves to the next slide when the commentary ends; off – the slide stays
- F full screen · Esc slide overview
- Alt + click (Linux: Ctrl + click) – zoom into a spot on the slide; click again to zoom out
- Underlined abbreviations – hover the mouse over them for an explanation

About me



Martin Necpal – university lecturer, Institute of Production Technology
Faculty of Materials Science and Technology in Trnava, STU Bratislava

- **PhD (2010):** automated process planning and integration into CAx systems
- **CAD / CAM** applications and NC programming
- **FEM** Modeling and simulation machining and forming processes
- **Laser machining** on the 5-axis LASERTEC 80 Shape – teaching, theses, research
- **Python, MATLAB, data science, AI / machine learning** in practice

✉ martin.necpal@stuba.sk · 🌐 martinnecpal.aiplaces.org · ORCID 0000-0003-2142-9204 · GitHub: martinnecpal

1 · Introduction to GAMS

General Algebraic Modeling System – what it is, where it is used, how to install and run it



About GAMS – www.gams.com (© GAMS Development Corp.)

What is GAMS?

- A **high-level algebraic modelling language** for optimization
- You write the model **as mathematics**: sets, parameters, variables, equations
- GAMS **compiles** the model and passes it to a **solver**
- Same model → **different solvers** without rewriting
- Linear, nonlinear, discrete, stochastic, equilibrium – at large scale

Key idea
separate **what** the problem is (the model)
from **how** it is solved (the solver)

GAMS Ecosystem

The GAMS ecosystem

GAMS Language today
Algebraic modelling language + GAMS Studio IDE

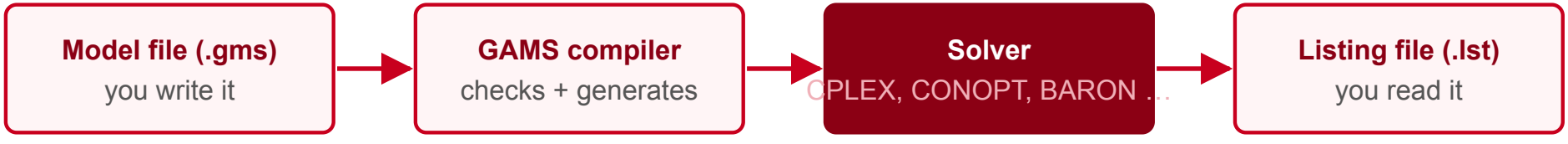
GAMSPy today
Python-native interface – the same modelling power inside Python

GAMS MIRO
Turns a model into an interactive web application for non-experts

GAMS Engine
Runs optimization jobs at scale – parallel, cloud, large memory

GAMS Workflow

How GAMS works



sets, data, variables, equations

syntax check, model instance

numerical algorithm

solver / model status, variable values

Sets → Data (Parameters, Tables, Scalars) → Variables → Equations → Model → Solve → Display

Problem types and solvers

Model type	Meaning	Typical solvers
LP	Linear programming	CPLEX, HiGHS, Gurobi, XPRESS
MIP	Mixed-integer linear	CPLEX, HiGHS, Gurobi, SCIP
NLP	Nonlinear programming	CONOPT, IPOPT, SNOPT, KNITRO
MINLP	Mixed-integer nonlinear	DICOPT, SBB, SHOT, BARON
QCP / **MIQCP**	Quadratic constraints	CPLEX, Gurobi, MOSEK
MCP / **MPEC**	Equilibrium / complementarity	PATH, NLPEC
Global	Proven global optimum	BARON, SCIP

> Switching the solver = one line: `option NLP = BARON;`

GAMS Applications

Where is GAMS used?

⚡ **Energy systems**
power planning, unit commitment, CCS

🏛️ **Economics & policy**
CGE models, trade (OECD METRO)

🌾 **Agriculture, water, forestry**
resource allocation, water networks

📈 **Finance**
portfolio and risk optimization

🚚 **Logistics & operations**
transport, scheduling, routing

⚙️ **Engineering**
chemical processes, design, manufacturing

Case studies on gams.com: OECD, TotalEnergies (CCS), Alpro, Rhodium Group

GAMS in chemical engineering

- **Process synthesis** – flowsheet structure (MINLP)
- **Heat exchanger networks** – heat integration
- **Distillation** – sequencing, feed tray
- **Reactor networks & kinetics**
- **Pooling and blending** – refineries
- **Batch plants** – design, scheduling

Examples in the GAMS Model Library

`process`	alkylation
`procsel`	flowsheet selection
`synheat`	HEN synthesis
`feedtray`	feed plate location
`hda`	toluene hydrodealkylation
`batchdes`	batch plant design
`haverly`, `pool`	pooling
`methanol`, `gasoil`	kinetics

Classic field of application (Grossmann, CMU, and many others)

GAMS in mechanical & manufacturing engineering

- **Machine elements** – gears, springs, bearings
- **Structural optimization** – trusses, shapes
- **Optimal control** – robot arm, trajectories
- **Manufacturing** – cutting stock, production planning, flow shop
- **Energy in production** – machine scheduling

Examples in the GAMS Model Library

`gear`	gear train design
`spring`	coil compression spring
`bearing`, `jbearing`	thrust / journal bearing
`trussm`	truss topology
`camshape`	cam shape
`robot`	robot arm
`trimloss`, `cutstock`	cutting
`flowshop`, `prodplan`, `rcpsp`	scheduling

Why should engineers use GAMS?

- The model reads like the **mathematical formulation** on paper
- **Same model, many solvers** – compare local vs. global results
- Scales from **classroom examples to industrial models**
- **450+ ready models** in the GAMS Model Library
- Free **demo license** with every download; academic licenses available
- With **GAMSPy** the model lives inside Python – data, plots, web apps

➡ Next: one example, first in **GAMS**, then in **GAMSPy**

Resources

- GAMS homepage – www.gams.com
- GAMS documentation – gams.com/latest/docs
- GAMS Model Library – gams.com/latest/gamslib_ml/libhtml
- GAMSPy documentation – gamspy.readthedocs.io
- GAMSPy examples – github.com/GAMS-dev/gamspy-examples
- Brooke, Kendrick, Meeraus, Raman: *GAMS – A User's Guide*

GAMS Studio Installation

Installing GAMS

- **1. Download** – gams.com/download · Windows, Linux, macOS
- **2. Install** – **GAMS Studio is installed automatically**; SmartScreen → *More info* → *Run anyway*
- **3. License** – free **demo license** (size-limited) included; free / academic licenses at gams.com/free-licenses
- In Studio: *Help* → *GAMS Licensing* to paste or check the license



GAMS download page (© GAMS Development Corp.)

GAMS Studio The IDE

GAMS Studio at a glance



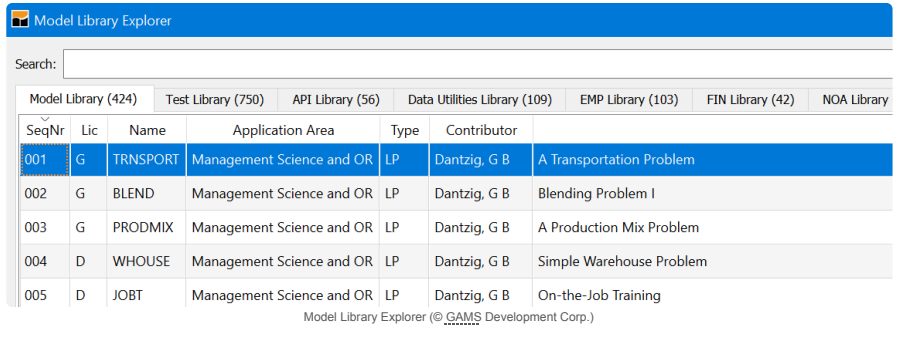
GAMS Studio main window (© GAMS Development Corp.)

Area	Purpose
Project Explorer	Project files; main file marked with green ▶
Code Editor	`.gms` with highlighting and completion
Listing file	`.lst` – solve summary, equations, variables
Process Log	Run messages, errors, clickable links
Toolbar	New · Open · Model Library · **Run (F9)**

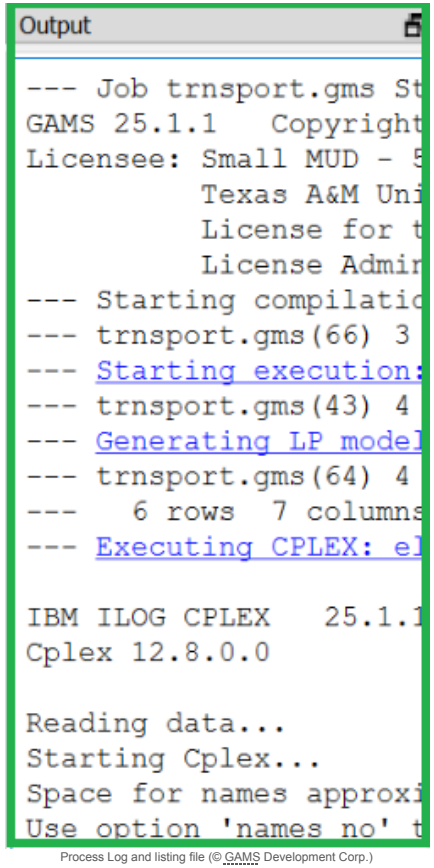
GAMS Studio First run

Your first run in 5 steps

1. **Create a project** – *File* → *New Project…*
2. **Open a model** – *GAMS* → *Model Library Explorer* (**F6**) → `trnsport`, or *File* → *New…*
3. **Check the main file** – green ▶ in the Project Explorer (*right-click* → *Set as Main File*)
4. **Run** – green ▶ button or **F9**
5. **Read the results** – Process Log + `.lst` opens automatically



Reading the output



Process Log and listing file (© GAMS Development Corp.)

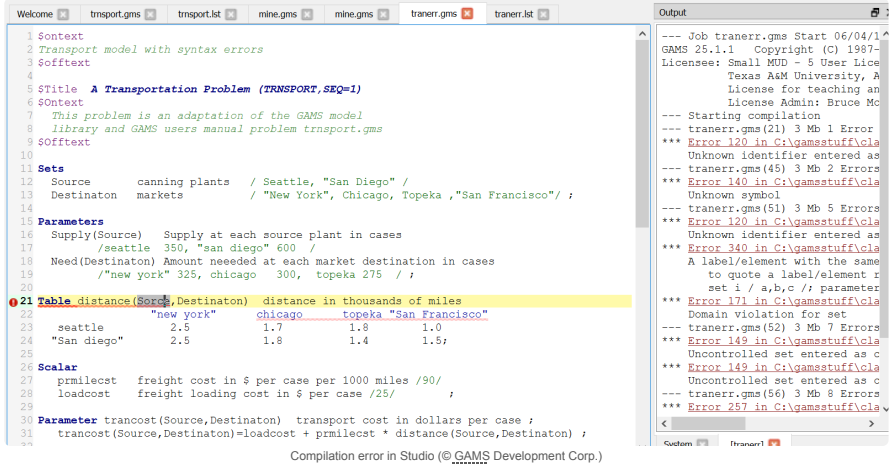
Log line	Double-click opens
🔵 Blue	the `.lst` file at that place
🔴 Red	**error** – the line in the source
🟢 Green	created files (GDX, REF, PUT)

Listing tree: Compilation · Equation listing · Model statistics · **Solution report** · Display
Look first at `SOLVER STATUS 1` · `MODEL STATUS 1 Optimal` · `OBJECTIVE VALUE`

GAMS Studio Errors

Finding and fixing errors

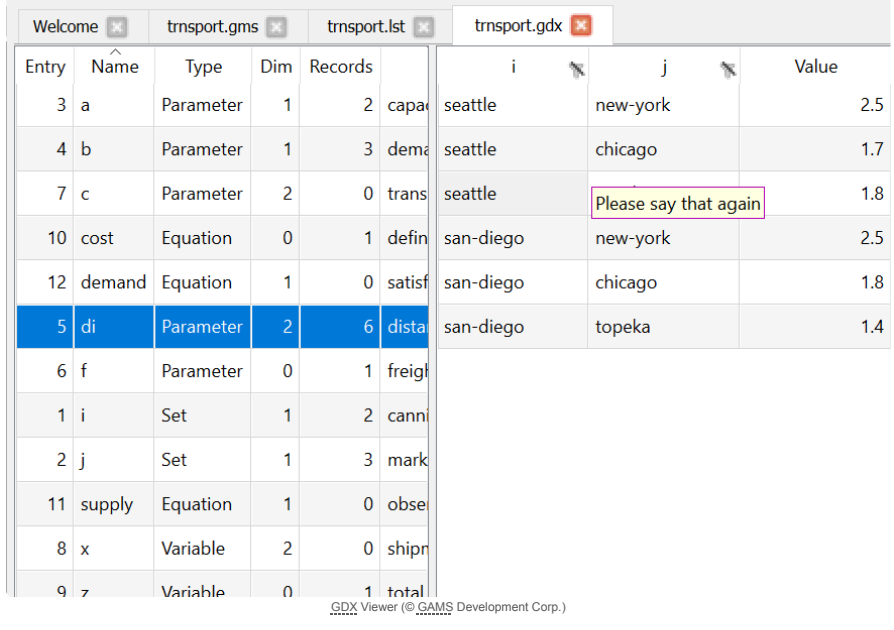
- Errors appear as **red lines** in the Process Log
- **Always start with the first error** – later ones are often consequences
- Click the red line → cursor jumps to the error in the `.gms`
- In the `.lst` the position is marked with `$` + error number
- Typical: missing `;`, undeclared symbol, misspelled element, `=` instead of `=e=`



Compilation error in Studio (© GAMS Development Corp.)

More useful Studio tools

- **GDX Viewer** – sets, parameters, solutions as tables; export to Excel
- **Model Library Explorer** (F6) – 400+ ready-to-run models
- **Solver option editor** – `conopt.opt`, `cplex.opt` with a form
- **Integrated help** (F1) – docs for the keyword under the cursor
- **Debugger** – breakpoints, inspect data
- **NEOS / GAMS Engine** – solve remotely



GAMS Studio Shortcuts

Handy shortcuts

Action	Windows / Linux	macOS
Run model	**F9**	**F9**
Compile only	Shift + F9	Shift + F9
Model Library	F6	F6
Help on keyword	F1	F1
Code completion	Ctrl + Space	Ctrl + Space
Comment / uncomment	Ctrl + *	Cmd + *
Matching bracket	Ctrl + B	Cmd + B
Navigator (file / line)	Ctrl + K	Ctrl + K

➡ Next: our first example written in GAMS

Links

- Download – gams.com/download
- Installation & licensing – gams.com/latest/docs/UG_MAIN.html#UG_INSTALL
- Free licenses – gams.com/free-licenses
- GAMS Studio manual – gams.com/latest/docs/T_STUDIO.html
- GAMS Studio tutorial (B. McCarl) – gams.com/latest/docs/UG_studio_tutorial.html

2 · Example 1 in GAMS

A simple linear program – graphical solution vs. GAMS solution

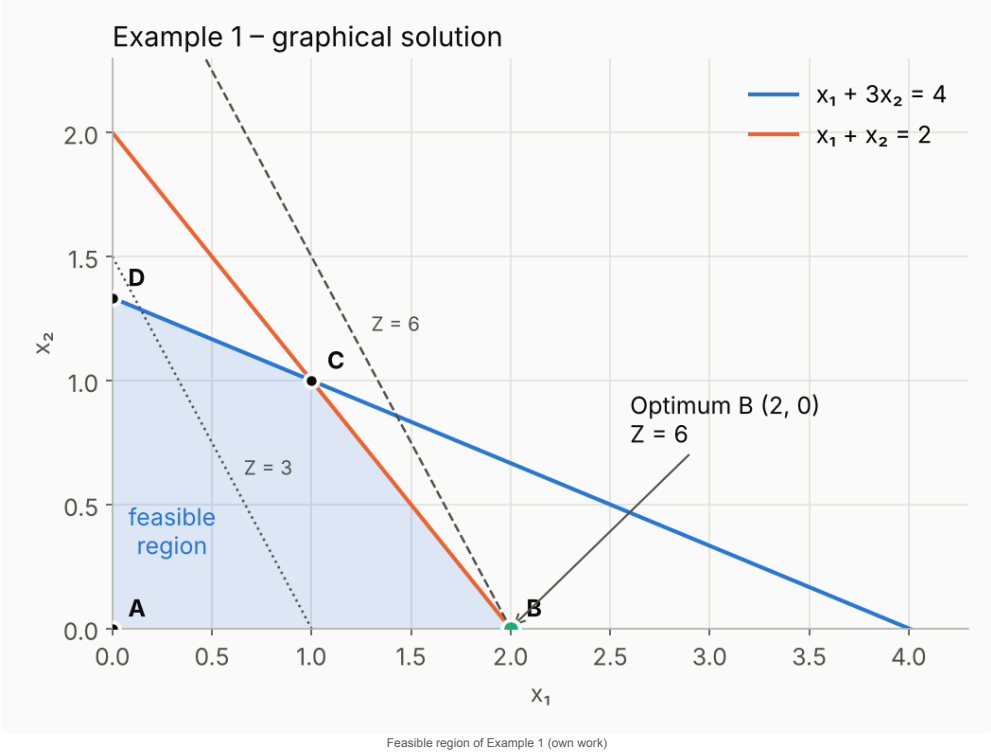
Example adapted from Prof. Elvis Ahmetović, University of Tuzla

Example 1 Problem

The problem

$$\max Z = 3x_1 + 2x_2$$
$$\text{s.t. } x_1 + 3x_2 \leq 4$$
$$x_1 + x_2 \leq 2$$
$$x_1,\ x_2 \geq 0$$

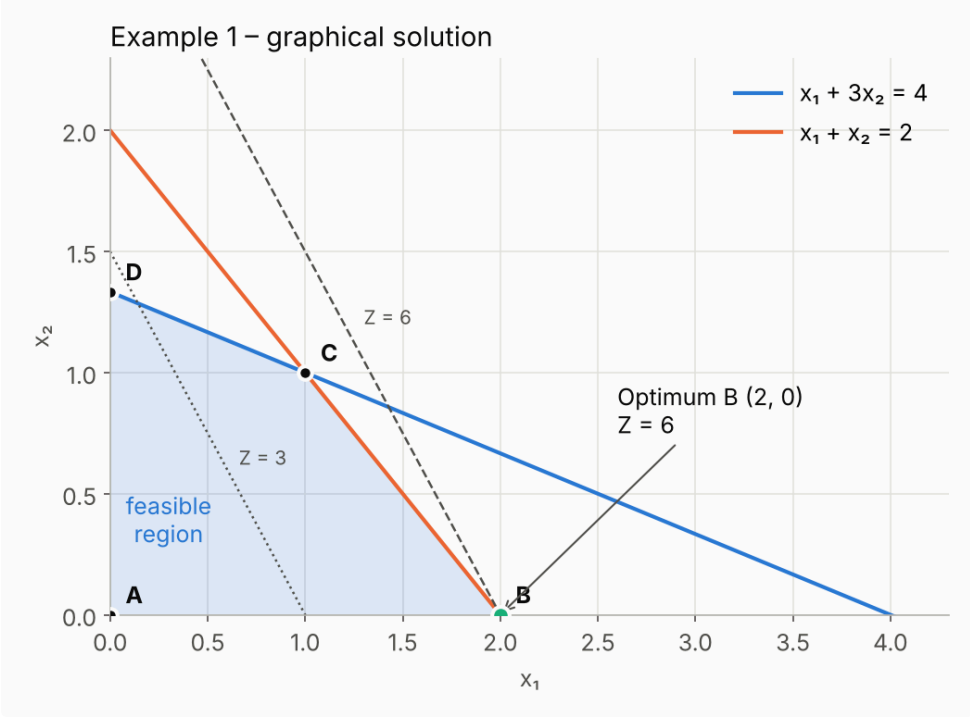
Two variables → solve it **graphically**, then check the result **in GAMS**.



Feasible region of Example 1 (own work)

Example 1 Graphical solution

Graphical solution



Feasible region = polygon **A B C D**; the optimum of an LP lies in a **corner point**.

Corner	(x_1, x_2)	$Z = 3x_1 + 2x_2$
A	(0, 0)	0
B	**(2, 0)**	**6** ✅
C	(1, 1)	5
D	(0, 4/3)	8/3

C: $x_1 + 3x_2 = 4$ and $x_1 + x_2 = 2 \rightarrow x_2 = 1, x_1 = 1$

Optimum: $x_1 = 2, x_2 = 0, Z = 6$

Example 1 GAMS code

The same model in GAMS

```
$Title Simple LP optimisation problem

Positive Variables x1, x2;
Free Variable      Z;

Equations obj, eq1, eq2;

obj..  Z =e= 3*x1 + 2*x2;
eq1..  x1 + 3*x2 =l= 4;
eq2..  x1 +   x2 =l= 2;

Model Example01 /all/;
Solve Example01 maximizing Z using LP;

Display x1.l, x2.l, Z.l;
```

- **Variables** – x_1, $x_2 \geq 0$, objective Z free
- **Equations** – declared, then defined with `..`
- **Model** – all equations
- **Solve** – direction, objective, model type
- **Display** – print the results

Example 1 GAMS code

Reading the code line by line

Statement	Meaning
`Positive Variables x1, x2;`	Decision variables with bound x ≥ 0
`Free Variable Z;`	Objective variable – must be **free**
`Equations obj, eq1, eq2;`	**Declare** the equation names
`obj.. Z =e= …;`	**Define** the equation; `..` separates name and body
`=e= =l= =g=`	equal · less or equal (≤) · greater or equal (≥)
`Model Example01 /all/;`	Model = all declared equations
`Solve … maximizing Z using LP;`	Direction, objective variable, model type
`Display x1.l, …;`	Print the **level** (`.l`) = optimal value

Every statement ends with `;` – the most common beginner error!

Example 1 Live in GAMS Studio

Running it in GAMS Studio LIVE

1. *File* → *New Project…* → folder e.g. `GAMS_lecture`
2. *File* → *New…* → `Example01.gms`, type or paste the code
3. Save (**Ctrl + S**) – check the green ▶ main-file marker
4. Press **F9** (or the green Run button)
5. The Process Log shows the run; `Example01.lst` opens automatically

> Live: first delete one `;` → red error line + `$` marker in the listing, then fix it.

Example 1 Results

Results in the listing file

```
             S O L V E     S U M M A R Y

     MODEL   Example01      OBJECTIVE  Z
     TYPE    LP             DIRECTION  MAXIMIZE
     SOLVER  CPLEX          FROM LINE  18

**** SOLVER STATUS     1 Normal Completion
**** MODEL STATUS      1 Optimal
**** OBJECTIVE VALUE           6.0000
```

```
----   20 VARIABLE x1.L  =  2.000
          VARIABLE x2.L  =  0.000
          VARIABLE Z.L   =  6.000
```

✅ **Same as the graphical solution: $x_1 = 2$, $x_2 = 0$, $Z = 6$**

Default LP solver: **CPLEX**

Example 1 Exercise

Try it yourself

- Change the objective to $\mathbf{Z = 2x_1 + 3x_2}$ – which corner wins now?
- Look at the **marginals** `eq1.m`, `eq2.m` – which constraint is binding?
- Switch the solver: add `option LP = HiGHS;` before `Solve`
- Change `maximizing` to `minimizing` – what happens and why?

➡ Next: the same model in **Python with GAMSPy**

3 · From GAMS to GAMSPy

Installing Python, GAMSPy and Visual Studio Code on Windows

What is GAMSPy?

- A **Python package** for algebraic modelling – sets, parameters, variables, equations as Python objects
- Uses the **GAMS engine and solvers** in the background
- **No separate GAMS installation** – `pip install gamspy` brings its own GAMS core
- Works with **pandas, NumPy, Matplotlib, Jupyter, web apps**
- Free **demo license**; **free for academics** (GAMS Academic Program)

⚠ GAMS and GAMSPy licenses are **different** – a GAMS license cannot be used for GAMSPy.

GAMSPy Installation

Step 1 – Install Python

- GAMSPy needs **Python 3.11 or newer**
- Download from python.org/downloads
- In the installer tick ✅ **"Add python.exe to PATH"**
- Check in **PowerShell**:

```
python --version
```

Step 2 – Create a virtual environment

Open **PowerShell** in your working folder (e.g. `C:\GAMS_lecture`):

```
python -m venv .gamspy_venv
.gamspy_venv\Scripts\Activate.ps1
```

The prompt now starts with `(.gamspy_venv)` ✅

⚠ PowerShell blocks the script ("running scripts is disabled")?

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Step 3 – Install GAMSPy

```
pip install gamspy
```

Check the installation:

```
gamspy --version
```

```
GAMSPy version: 1.29.0
GAMS version: 54.6.0
gamspy_base version: 54.6.0
```

Update later with `pip install gamspy --upgrade`

GAMSPy Installation

Step 4 – Solvers

Show the installed solvers:

```
gamspy list solvers
```

Included by default: **CPLEX** (LP, MIP), **CONOPT4** (NLP), **SBB** (MINLP), **PATH** (MCP)

Install more on demand:

```
gamspy list solvers --all
gamspy install solver highs ipopt
```

Choose the solver in Python:
`model.solve(solver="highs")`

Step 5 – License (optional)

The **demo license** works out of the box for small models – enough for this lecture.
With an academic or full license (36-character access code):

```
gamspy install license <access_code>
gamspy show license
```

Back to demo: `gamspy uninstall license` · Demo limits: GAMS documentation

GAMSPy Installation

Step 6 – Test it

```python
import gamspy as gp

m = gp.Container()
x = gp.Variable(m, "x")
e = gp.Equation(m, "e")
e[...] = x >= 1

model = gp.Model(m, "test", equations=[e],
                 problem="LP", sense="min", objective=x)
model.solve()

print(model.status, model.objective_value)
```

Save as `test_gamspy.py`, run `python test_gamspy.py`:

```
ModelStatus.OptimalGlobal 1.0
```

✅ **GAMSPy is ready**

GAMSPy Installation

Quick reference

Task	Command
Create environment	`python -m venv .gamspy_venv`
Activate (PowerShell)	`.gamspy_venv\Scripts\Activate.ps1`
Install GAMSPy	`pip install gamspy`
Version	`gamspy --version`
List solvers	`gamspy list solvers`
Add a solver	`gamspy install solver <name>`
Install license	`gamspy install license <access_code>`
Show license	`gamspy show license`

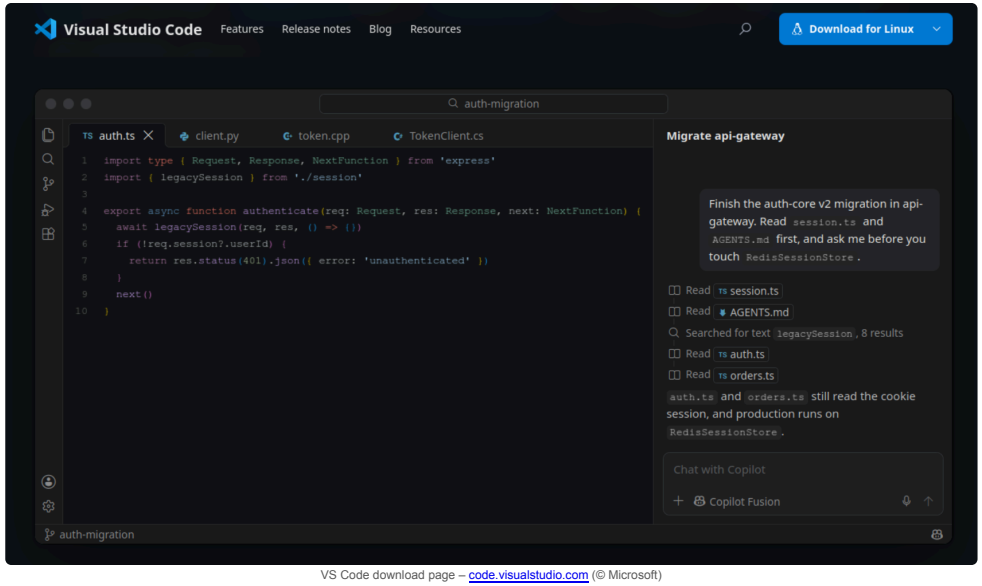
Docs: gamspy.readthedocs.io – Installation and Licensing

VS Code Setup

Step 1 – Install VS Code

1. Download from code.visualstudio.com → **Download for Windows**
2. Run the installer and tick:
   ✅ **Add "Open with Code"** to the Explorer context menu · ✅ **Add to PATH**
3. Start VS Code

> VS Code is **free**, light and runs on Windows, macOS and Linux.



VS Code download page – code.visualstudio.com (© Microsoft)

VS Code Setup

Step 2 – Python extension

1. Open **Extensions** (**Ctrl + Shift + X**)
2. Search **Python** → install the extension by **Microsoft** (`ms-python.python`)
3. The **Python Debugger** is installed automatically with it

Optional: **Jupyter** extension (Microsoft) for notebooks (`.ipynb`)

> ⚠ The extension does **not** include Python itself – Python 3.11+ must be installed first.

VS Code Setup

Step 3 – Open your project folder

- *File* → *Open Folder…* → e.g. `C:\GAMS_lecture`
- or right-click the folder in Windows Explorer → **Open with Code**
- Confirm **"Yes, I trust the authors"**

> One folder = one project – your `.py` files and `.gamspy_venv` live here.

VS Code Setup

Step 4 – Select the Python environment

Command Palette **Ctrl + Shift + P**:

A) Environment exists (created in PowerShell)
→ `Python: Select Interpreter` → choose `.gamspy_venv`

B) Create a new one in VS Code
→ `Python: Create Environment` → **Venv** → Python 3.11+

C) Create it in the terminal
→ `python -m venv .gamspy_venv`
(macOS / Linux: `python3`) → then as in A)

System	Activate	Deactivate
Windows – PowerShell	`.gamspy_venv\Scripts\Activate.ps1`	
Windows – cmd	`.gamspy_venv\Scripts\activate.bat`	`deactivate`
macOS / Linux	`source .gamspy_venv/bin/activate`	

The selected environment is shown in the **status bar** (bottom right).

VS Code Setup

Step 5 – Install GAMSPy in the environment

Integrated terminal: **Terminal** → **New Terminal** (**Ctrl + Shift +** `)
The prompt should start with the environment name, e.g. `(.gamspy_venv)` ✅

```
pip install gamspy matplotlib
gamspy --version
```

We install **Matplotlib** now as well – for plotting results later.

VS Code Setup

Step 6 – Run your first script

Create `test_gamspy.py`, paste the test model, save (**Ctrl + S**):

- Click ▶ **Run Python File** (top right of the editor), or
- Right-click → *Run Python* → *Run Python File in Terminal*
- Select lines + **Shift + Enter** → run only the selection

Output in the **Terminal** panel:

```
ModelStatus.OptimalGlobal 1.0
```

VS Code Shortcuts

VS Code shortcuts for this lecture

Action	Shortcut
Command Palette	**Ctrl + Shift + P**
Extensions	Ctrl + Shift + X
New terminal	Ctrl + Shift + `
Run selection in terminal	Shift + Enter
Debug (start)	F5
Comment / uncomment line	Ctrl + /
Quick open file	Ctrl + P

➡ Next: Example 1 written in **GAMSPy**

4 · Example 1 in GAMSPy

Same model – now in Python

GAMSPy Example 1

GAMS → GAMSPy: the dictionary

GAMS	GAMSPy
(the whole .gms file)	`m = gp.Container()`
`Set i / i1, i2 /;`	`i = gp.Set(m, "i", records=["i1", "i2"])`
`Parameter p(i) / … /;`	`p = gp.Parameter(m, "p", domain=i, records=…)`
`Positive Variable x;`	`x = gp.Variable(m, "x", type="positive")`
`Free Variable Z;`	`Z = gp.Variable(m, "Z", type="free")`
`Equation e;`	`e = gp.Equation(m, "e")`
`e.. lhs =e= rhs;`	`e[...] = lhs == rhs`
`=l=` / `=g=`	`<=` / `>=`
`sum(i, a(i))`	`gp.Sum(i, a[i])`
`Model mod /all/;`	`gp.Model(m, "mod", equations=m.getEquations(), …)`
`Solve mod using LP maximizing Z;`	`problem="LP", sense="max", objective=Z` + `mod.solve()`
`Display x.l;`	`print(x.toValue())` or `print(x.records)`

GAMSPy Example 1

Side by side

```
$title Simple LP optimisation problem

Positive variables x1,x2;
Free Variable Z;

Equations obj,eq1,eq2;

obj..Z=e=3*x1+2*x2;
eq1..x1+3*x2=l=4;
eq2..x1+x2=l=2;

Model Primjer01 /all/;
Solve Primjer01 maximizing Z using LP;

Display x1.l,x2.l,Z.l;
```

GAMS Studio – `Example01.gms`

```python
"""Simple LP optimisation problem (GAMSPy translation of Example01.gms)."""

import sys
import gamspy as gp

# gams container definition
m = gp.Container()

# variable definition
x1 = gp.Variable(m, name="x1", type="positive")
x2 = gp.Variable(m, name="x2", type="positive")
z = gp.Variable(m, name="z", type='free')

# equation definition
eq1 = gp.Equation(m, name="eq1")
eq2 = gp.Equation(m, name="eq2")
eq1[...] = x1 + 3 * x2 <= 4
eq2[...] = x1 + x2 <= 2

# objective definition
objective = gp.Equation(m, "obj")
objective[...] = z == 3 * x1 + 2 * x2

primjer01 = gp.Model(
        m,
        name="Primjer01",
        equations = m.getEquations(),
        problem = "LP",
        sense = gp.Sense.MAX,
        objective = z,
    )

primjer01.solve(output=sys.stdout)

print("x1.l =", x1.toValue())
print("x2.l =", x2.toValue())
print("Z.l  =", primjer01.objective_value)
```

VS Code – `example01.py` (GAMSPy)

GAMSPy Example 1

How to convert – step by step

1. **Import and create a Container** – `import gamspy as gp`, `m = gp.Container()`
2. **Variables** – each declaration becomes `gp.Variable(...)` with `type=`
3. **Equations** – declare with `gp.Equation(...)`, define with `eq[...] = …`
4. **Relations** – `=e=` → `==`, `=l=` → `<=`, `=g=` → `>=`; no `;`
5. **Model** – list of equations + `problem`, `sense`, `objective`
6. **Solve** – `model.solve()` (default LP solver: CPLEX)
7. **Results** – `.toValue()` for scalars, `.records` (pandas DataFrame) for indexed symbols

⚠ `eq1 = x1 + 3*x2 <= 4` without `[...]` overwrites the Python variable instead of defining the equation.

GAMSPy Example 1

Run it in VS Code LIVE

1. Open the folder `C:\GAMS_lecture` – environment `.gamspy_venv` selected
2. New file `example01.py` – paste the code, **Ctrl + S**
3. Click ▶ **Run Python File**
4. Read the result in the **Terminal** panel

Full solver log as in GAMS Studio:

```
import sys
example01.solve(output=sys.stdout)
```

GAMSPy Example 1

Results

`solve()` returns a summary table (pandas DataFrame):

```
      Solver Status   Model Status  Objective  ...  Model Type  Solver  Solver Time
0  NormalCompletion  OptimalGlobal        6.0  ...          LP   CPLEX        0.014
```

Values:

```
x1 = 2.0   x2 = 0.0   Z = 6.0
```

Full record – `print(x1.records)` :

```
   level  marginal  lower  upper  scale
0    2.0       0.0    0.0    inf    1.0
```

✅ **Same as graphical solution and GAMS: $x_1 = 2$, $x_2 = 0$, $Z = 6$**

GAMSPy Example 1

The Python way – shorter

The objective is written **directly as an expression** – no `Z` variable, no `obj` equation:

```
example01 = gp.Model(m, "example01",
    equations=[eq1, eq2],
    problem="LP", sense="max",
    objective=3 * x1 + 2 * x2)
example01.solve()

print("Model status :", example01.status)
print("Objective Z  :", example01.objective_value)
print("Marginals    :", eq1.records["marginal"][0],
                        eq2.records["marginal"][0])
```

```
Model status : ModelStatus.OptimalGlobal
Objective Z  : 6.0
Marginals    : 0.0 3.0
```

eq2 is **binding** (shadow price 3), eq1 has slack (0).

GAMSPy Example 1

Bonus: back to GAMS and LaTeX

A GAMSPy model can be exported:

```
example01.toGams("exported")  # exported/example01.gms
example01.toLatex("latex")    # latex/example01.tex
```

Open the model in GAMS Studio, or paste the equations into a paper.

➡ Next: more realistic data – Example 3

Generated `example01.gms`:

```
positive Variable x1;
positive Variable x2;
free Variable Z;
...
obj .. Z =e= 3 * x1 + 2 * x2;
eq1 .. x1 + 3 * x2 =l= 4;
eq2 .. x1 + x2 =l= 2;
solve example01 using LP MAX Z;
```

5 · Example 3 – Mixing of streams

Sets, parameters and data in GAMSPy

Example after Prof. Elvis Ahmetović; problem from M. R. Eden (2011)

Example 3 Problem

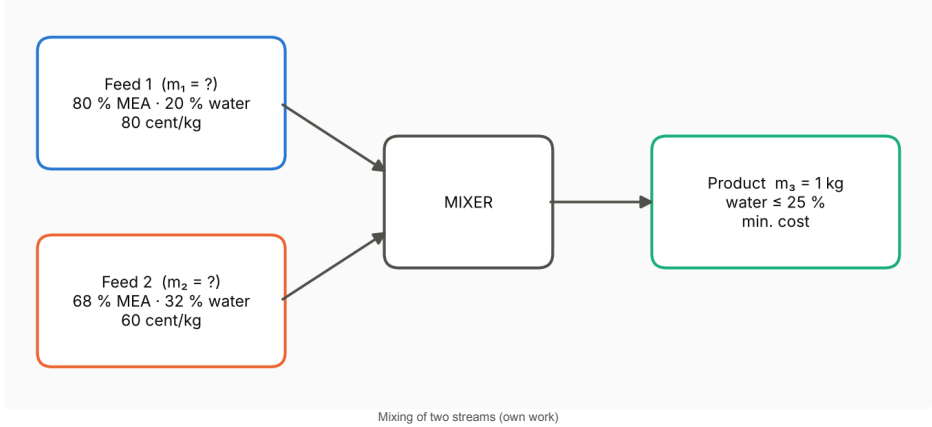
The problem

Two feed streams are mixed into an **MEA / water mixture**.

Feed	MEA	Water	Cost
Feed 1	80 %	20 %	80 cent/kg
Feed 2	68 %	32 %	60 cent/kg

Product: max. **25 mass.% water**

Which mixing ratio gives the **minimum cost** per 1 kg of mixture?



Mixing of two streams (own work)

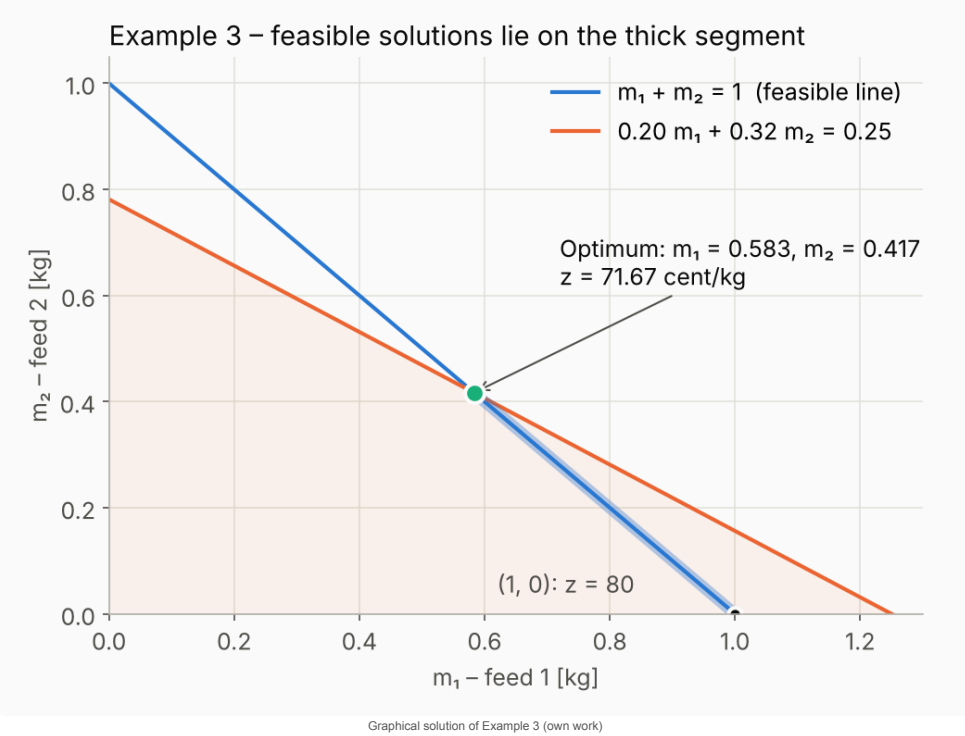
Example 3 Model

Mathematical formulation

- m_1, m_2 – mass of feed 1 and feed 2 [kg] (basis 1 kg)
- z – cost of 1 kg mixture [cent]

$$\begin{aligned} \min\ & z = 80\,m_1 + 60\,m_2 \\ \text{s.t. } & 0.20\,m_1 + 0.32\,m_2 \le 0.25 \\ & m_1 + m_2 = 1 \\ & m_1,\ m_2 \ge 0 \end{aligned}$$

Graphically: $m_1 = 0.583$, $m_2 = 0.417$, $z = 71.67$ cent/kg



Graphical solution of Example 3 (own work)

Example 3 Step 1

Step 1 – direct translation

```python
import gamspy as gp

# gams container definition
m = gp.Container()

# variable definition
m1 = gp.Variable(m, "m1", type="positive", description="mass of feed 1 [kg]")
m2 = gp.Variable(m, "m2", type="positive", description="mass of feed 2 [kg]")

# equations definition
water   = gp.Equation(m, "water")
balance = gp.Equation(m, "balance")
water[...]   = 0.20 * m1 + 0.32 * m2 <= 0.25
balance[...] = m1 + m2 == 1

mixing = gp.Model(m, "mixing", equations=m.getEquations(),
                  problem="LP", sense="min", objective=80 * m1 + 60 * m2)
mixing.solve()

print("Z.l  =", mixing.objective_value)
print("m1=", m1.toValue())
print("m2=", m2.toValue())
```

VS Code – `mixing_of_streams.py` (GAMSPy)

```
z  = 71.667 cent/kg
m1 = 0.583 kg
m2 = 0.417 kg
```

✅ **Same pattern as Example 1**

New: `==` for the balance and `description=` texts.

Example 3 Model vs. data

Problem: numbers are inside the equations

- Costs and compositions are **hard-coded** – a 3rd feed or component → rewrite the model
- Real models: **hundreds of streams**, data from Excel / databases → **separate model and data**

Concept	GAMS	GAMSPy
Index set	`Set f /feed1, feed2/;`	`gp.Set(m, "f", records=[...])`
Data	`Parameter cost(f) /.../;`	`gp.Parameter(m, "cost", domain=f, records=...)`
Table	`Table w(f,c)`	`gp.Parameter(..., domain=[f, c], records=DataFrame)`
Indexed variable	`x(f)`	`gp.Variable(m, "x", domain=f)`
Sum	`sum(f, cost(f)*x(f))`	`gp.Sum(f, cost[f] * x[f])`

Example 3 Step 2

Step 2 – general (indexed) formulation

- **Sets:** f – feeds, c – components
- **Parameters:** $cost_f$, $w_{f,c}$ mass fraction, $wmax_c$ limit, M product amount
- **Variable:** $x_f \geq 0$ – mass of feed f in product

$$\min \sum_f cost_f \, x_f$$

$$\text{s.t.} \sum_f w_{f,c} \, x_f \leq wmax_c \, M \quad \forall c \text{ with a limit}$$

$$\sum_f x_f = M$$

➡ The model **does not change** when you add feeds or components – only the data.

Example 3 Step 2

Sets and data

```
import pandas as pd
import gamspy as gp

m = gp.Container()

# Sets
f = gp.Set(m, "f", records=["feed1", "feed2"], description="feed streams")
c = gp.Set(m, "c", records=["MEA", "water"],  description="components")

# Data — composition as a pandas DataFrame (could come from Excel)
composition = pd.DataFrame(
    [["feed1", "MEA", 0.80], ["feed1", "water", 0.20],
     ["feed2", "MEA", 0.68], ["feed2", "water", 0.32]],
    columns=["f", "c", "value"])

w    = gp.Parameter(m, "w", domain=[f, c], records=composition)
cost = gp.Parameter(m, "cost", domain=f, records=[["feed1", 80], ["feed2", 60]])
wmax = gp.Parameter(m, "wmax", domain=c, records=[["water", 0.25]])
M    = gp.Parameter(m, "M", records=1)
```

Example 3 Step 2

Variables, equations, model

```
x = gp.Variable(m, "x", type="positive", domain=f)

spec    = gp.Equation(m, "spec", domain=c)
balance = gp.Equation(m, "balance")

# one equation for every component c that has a limit
spec[c].where[wmax[c]] = gp.Sum(f, w[f, c] * x[f]) <= wmax[c] * M
balance[...]           = gp.Sum(f, x[f]) == M

mixing = gp.Model(m, "mixing", equations=m.getEquations(),
                  problem="LP", sense="min",
                  objective=gp.Sum(f, cost[f] * x[f]))
mixing.solve()
```

GAMSPy	GAMS equivalent
`spec[c]`	`spec(c)..` – an equation for each c
`.where[wmax[c]]`	`$wmax(c)` – only where a limit exists
`gp.Sum(f, …)`	`sum(f, …)`

Example 3 Results

Results

```
print(mixing.objective_value)
print(x.records[["f", "level"]])
```

```
Minimum cost: 71.667 cent/kg
       f     level
0  feed1  0.583333
1  feed2  0.416667
```

Check the product composition:

```
product = gp.Parameter(m, "product", domain=c)
product[c] = gp.Sum(f, w[f, c] * x.l[f]) / M
```

```
       c  value
0    MEA   0.75
1  water   0.25    ← constraint is active
```

Example 3 What-if

What-if: change the water limit LIVE

```
for limit in [0.20, 0.22, 0.25, 0.28, 0.32]:
    wmax.setRecords([["water", limit]])
    mixing.solve()
    x1, x2 = x.records["level"]
    print(limit, mixing.objective_value, x1, x2)
```

Change data → solve again, **no model rewriting**.

Water limit	Cost	Feed 1	Feed 2
0.20	80.000	1.000	0.000
0.22	76.667	0.833	0.167
0.25	**71.667**	**0.583**	**0.417**
0.28	66.667	0.333	0.667
0.32	60.000	0.000	1.000

➡ Next: turn this table into a **plot with Matplotlib**

Example 3 Exercise

Try it yourself

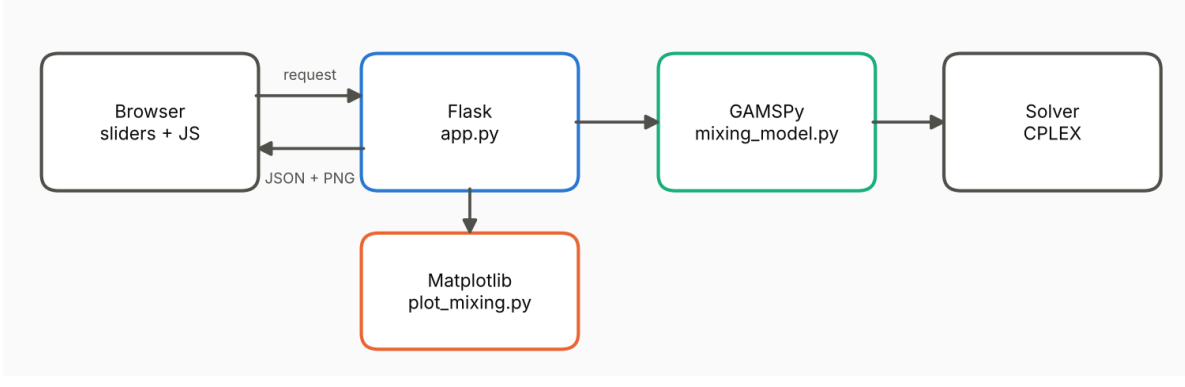
- Add a **third feed**: 90 % MEA, 10 % water, 95 cent/kg – only the data changes!
- Add a **minimum MEA content** of 72 % – hint: a `wmin(c)` and a `>=` equation
- Produce **M = 1000 kg** instead of 1 kg
- Water limit **0.15**? Check `mixing.status`

6 · From model to web page

GAMSPy + Matplotlib + Flask – in 7 testable steps

Web app Architecture

What we are going to build



File	Step	Role
`mixing_model.py`	2	Example 3 as a Python function
`plot_mixing.py`	3	What-if chart with Matplotlib
`app_step4.py` … `app_step6.py`	4–6	Flask, growing step by step
`app.py` + `templates/index.html`	7	Final interactive page

Web app Step 1

Step 1 – Install Matplotlib and Flask

VS Code terminal, `.gamspy_venv` active:

```
pip install matplotlib flask
```

✅ **TEST**

```
python -c "import gamspy, matplotlib, flask; print('all OK')"
```

Folder structure:

```
C:\GAMS_lecture\webapp\
    mixing_model.py
    plot_mixing.py
    app_step4.py  app_step5.py  app_step6.py
    app.py
    templates\index.html
```

Step 2 – Model as a reusable function

`mixing_model.py` – Example 3 wrapped in a function: **inputs** → **results dictionary**

```python
def solve_mixing(water_limit=0.25, cost1=80.0, cost2=60.0, amount=1.0):
    m = gp.Container()                # new container for every call
    ...                               # sets, parameters, x(f), spec(c), balance
    mixing.solve()

    if "Optimal" not in mixing.status.name:
        return {"ok": False, "status": mixing.status.name}
    x1, x2 = x.records["level"]
    return {"ok": True, "cost": mixing.objective_value,
            "feed1": x1, "feed2": x2, ...}
```

✅ **TEST** – `python mixing_model.py`

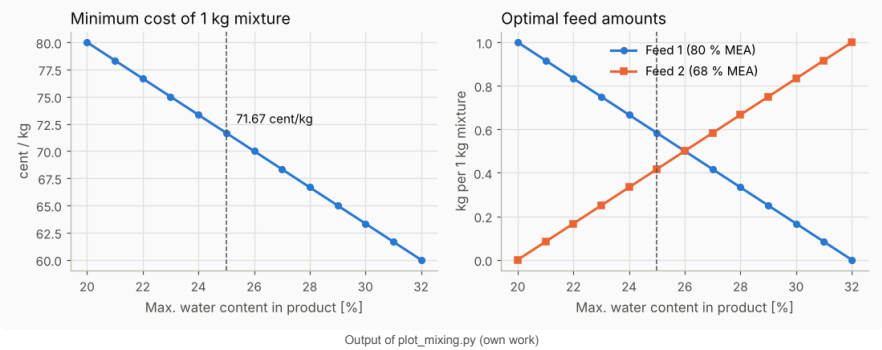
```
{'ok': True, 'status': 'OptimalGlobal', 'water_limit': 0.25, 'cost': 71.666..., 'feed1': 0.583..., ...}
{'ok': False, 'status': 'InfeasibleNoSolution', 'water_limit': 0.15}
```

Step 3 – What-if chart with Matplotlib

```python
import matplotlib
matplotlib.use("Agg")    # no window – draw to file / memory
import matplotlib.pyplot as plt

res = sweep(limits)      # 13 GAMSPy solves
ax1.plot(lim, [r["cost"]  for r in res])
ax2.plot(lim, [r["feed1"] for r in res], label="Feed 1")
ax2.plot(lim, [r["feed2"] for r in res], label="Feed 2")
fig.savefig("mixing_whatif.png")
```

✅ **TEST** – `python plot_mixing.py` → open `mixing_whatif.png`



Output of plot_mixing.py (own work)

Step 4 – The smallest Flask app

```python
# app_step4.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Hello from Flask!</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

✅ **TEST** – `python app_step4.py`, open **http://127.0.0.1:5000**

Stop the server with **Ctrl + C** before the next step.

- `@app.route("/")` – URL → Python function
- `debug=True` – restarts on every save

Step 5 – A form that solves the model

```
# app_step5.py – the browser sends a number, Flask calls GAMSPy
from flask import Flask, request, render_template_string
from mixing_model import solve_mixing

PAGE = """
<form method="post">
  Max. water content [%]: <input name="water" type="number" value="{{ water }}">
  <button>Solve</button>
</form>
{% if r.ok %} <p>Minimum cost: <b>{{ "%.2f"|format(r.cost) }} cent/kg</b></p>
{% else %}    <p>No feasible mixture ({{ r.status }})</p> {% endif %}
"""

@app.route("/", methods=["GET", "POST"])
def home():
    water = float(request.form.get("water", 25))
    return render_template_string(PAGE, water=water,
                                  r=solve_mixing(water_limit=water / 100))
```

✅ **TEST** – enter **22** → `76.67 cent/kg` ; enter **15** → *No feasible mixture*

Step 6 – Add the Matplotlib chart

`app_step6.py` – a second route returns the chart as a **PNG image**:

```python
import io
from flask import Response
from plot_mixing import make_figure
import matplotlib.pyplot as plt

@app.route("/plot.png")
def plot_png():
    water = float(request.args.get("water", 25))
    fig = make_figure(current=solve_mixing(water_limit=water / 100))
    buf = io.BytesIO()
    fig.savefig(buf, format="png")       # save into memory, not to disk
    plt.close(fig)                       # free memory
    return Response(buf.getvalue(), mimetype="image/png")
```

In the page: `<img src="/plot.png?water={{ water }}">`

✅ **TEST** – open **http://127.0.0.1:5000/plot.png?water=22**, then the main page

Step 7 – Interactive page: JSON API

```python
# app.py — two data routes + one page
@app.route("/")
def index():
    return render_template("index.html", feeds=FEEDS)

@app.route("/api/solve")                # results as JSON
def api_solve():
    water, cost1, cost2 = read_inputs()
    return jsonify(solve_mixing(water_limit=water, cost1=cost1, cost2=cost2))

@app.route("/plot.png")                 # chart as PNG (as in step 6)
def plot_png(): ...
```

✅ **TEST** – open **http://127.0.0.1:5000/api/solve?water=25&cost1=80&cost2=60**

```
{ "cost": 71.67, "feed1": 0.583, "feed2": 0.417, "ok": true,
  "status": "OptimalGlobal", "water": 0.25, "water_limit": 0.25 }
```

Step 7 – Interactive page: sliders + JavaScript

`templates/index.html` – sliders call the API on every change:

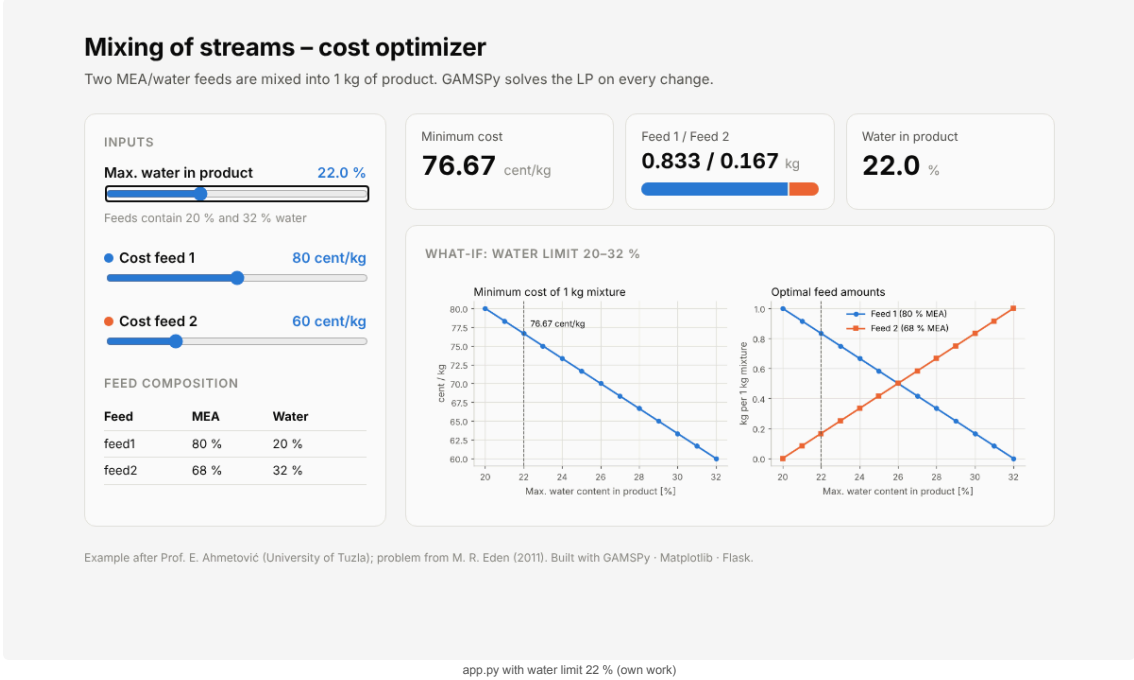
```
<input id="water" type="range" min="15" max="35" step="0.5" value="25">
<input id="cost1" type="range" min="40" max="120" value="80">
<input id="cost2" type="range" min="40" max="120" value="60">
<img id="plot">
```

```
async function update() {
  const q = new URLSearchParams({ water: water.value,
                                   cost1: cost1.value, cost2: cost2.value });
  const r = await (await fetch("/api/solve?" + q)).json();   // GAMSPy
  document.getElementById("cost").textContent = r.cost.toFixed(2);
  document.getElementById("plot").src = "/plot.png?" + q;    // Matplotlib
}
```

✅ **TEST** – `python app.py` , move the sliders – numbers and chart update

Web app Result

The result LIVE



app.py with water limit 22 % (own work)

- **Sliders:** water limit, cost of feed 1 and 2
- **Cards:** minimum cost, feed ratio, water in product
- **Chart:** what-if analysis, current case marked
- Infeasible input (< 20 % water) → red warning

Web app Summary

Summary & next steps

Layer	Tool	Our file
Optimization model	**GAMSPy**	`mixing_model.py`
Visualization	**Matplotlib**	`plot_mixing.py`
Web server	**Flask**	`app.py`
User interface	HTML + JavaScript	`templates/index.html`

- Third feed or minimum MEA content – only `mixing_model.py` changes
- Read feed data from Excel with `pandas.read_excel`
- Deploy for students – a university server, or GAMS MIRO for a ready-made UI

What we learned today

	GAMS	GAMSPy
Language	own algebraic language (`.gms`)	Python package
Environment	GAMS Studio	VS Code / Jupyter
Results	listing file `.lst` , GDX	pandas DataFrames, `.toValue()`
Data	tables in the model, GDX	pandas, Excel, databases
Apps	GAMS MIRO	Matplotlib, Flask, any Python tool
Solvers	the same: CPLEX, CONOPT, BARON, HiGHS, …	

Same algebraic model – two ways to write it. GAMS for pure modelling, GAMSPy when the model is part of a bigger Python workflow.

Review questions

1. What is the key idea of an algebraic modelling language like GAMS? answer
2. Why must the objective variable Z be declared `Free` in GAMS? answer
3. What do `=e=`, `=l=`, `=g=` mean and what are they in GAMSPy? answer
4. Which constraint is binding at the optimum of Example 1 and how do you see it? answer
5. What does `[...]` do in `eq1[...] = …` – and what happens without it? answer
6. Why is LP, not NLP, the correct model type for Example 3? answer
7. Why separate the model from the data with sets and parameters? answer
8. Why is a water limit below 20 % infeasible? answer

Where to continue

- GAMS Model Library – 400+ models, many from chemical and mechanical engineering
- GAMSPy examples – github.com/GAMS-dev/gamspy-examples
- GAMSPy for GAMS users – gamspy.readthedocs.io/…/gamspy_for_gams_users
- All code from today – ⬇ gamspy_lecture_code.zip

Homework: extend Example 3 with a third feed and a minimum MEA content, then show the result in the web app.

Acknowledgements

- Examples 1 and 3 adapted from the lecture of **Prof. Elvis Ahmetović** (University of Tuzla) – *Mathematical modelling in GAMS*, Erasmus+ lecture at STU, November 2025
- Example 3 problem from M. R. Eden: *Process Synthesis, Simulation and Optimization*, 2011
- Thanks to the hosts for the invitation

Thank you – questions?

Martin Necpal · martin.necpal@stuba.sk · martinnecpal.aiplaces.org

Sources and further reading

- Ahmetović, E.: *Mathematical modelling in General Algebraic Modelling System (GAMS).* Erasmus+ lecture, STU Bratislava, 2025
- Eden, M. R.: *Process Synthesis, Simulation and Optimization.* 2011
- Brooke, A.; Kendrick, D.; Meeraus, A.; Raman, R.: *GAMS – A User's Guide.* GAMS Development Corp.
- GAMS documentation and Model Library – gams.com/latest/docs
- GAMSPy documentation – gamspy.readthedocs.io
- VS Code Python tutorial – code.visualstudio.com/docs/python/python-tutorial
- Matplotlib – matplotlib.org · Flask – flask.palletsprojects.com

SCREENSHOTS OF GAMS STUDIO © GAMS DEVELOPMENT CORP., LINKED FOR TEACHING PURPOSES